Start here

This guide explains branch full-fst-lloh-wgd2-capped-x2 as reviewed on 23 October 2026, against implementation commit f73e81e. It is also the editable source of doc/full_fst_algorithms.pdf. Both can be read offline.

Read pages 2-10 first for the model and a worked example. Pages 11-18 explain why the search is exact for its stated objective and how it is accelerated. Pages 19-24 connect it to events, Python/C++, validation and the repository.

The central question is: which ancestor profiles explain two children at the lowest event cost? The program keeps all locally optimal profiles in a compact graph, then chooses representatives while walking down the tree.

Important scope: exact pair scores and exact joins over retained messages do not imply a globally optimal assignment of every ancestor on the whole tree. The bottom-up pass discards locally more expensive profiles.

Hint: first understand string, join and resolve. You can postpone C++ pointers, generated bound tables and graph-library internals.

Follow the data through the program

The input is a table of allele-specific copy numbers over genomic bins. The encoder makes aligned strings; X separates chromosome/allele blocks. A diploid normal is added when needed. The CLI uses the full model by default.

Workflow:

```
input table -> aligned copy-number strings
                    |
         supplied tree? -- yes --> use its topology
                    |
                   no
                    v
         pair distances -> FastME topology
                    |
                    v
         bottom-up messages -> top-down profiles
                    |
                    v
         branch scores -> optional event histories
```

A supplied topology skips distance-matrix construction. MEDICC NNI is optional and evaluates neighboring topologies with the ancestral backend.

Code: medicc/full_fst/core.py; medicc/full_fst/cli.py

Symbols, intervals and event costs

Each ordinary symbol is an integer CN from 0 through 8. X is a boundary, not an unknown CN. At the CLI, the existing input reader clips CNs above the chosen maximum. There is no hidden state for CN above 8 in this model.

A unit loss subtracts one over a contiguous interval, stopping at zero. A unit gain adds one to positive CNs; zero stays zero. Gains cannot resurrect a lost allele. An interval can continue through zero-copy bins, but never across X. Each unit interval costs one, independent of its number of bins.

Examples, with fixed parent and child:

```
22  -> 11     one loss covering both bins: cost 1
2X2 -> 1X1    two separate losses:          cost 2
0   -> 1      no permitted directed history
```

Thus this is not a sum of absolute CN differences. Segment structure matters. A biological interpretation is constrained by this event model; a minimum-cost history is not evidence that its individual events occurred uniquely.

Code: medicc/full_fst/events.py::replay; native assets/ancestral_parts.txt

The capped whole-genome duplication model

Write A = L_LOH G: a loss/LOH stage followed by a gain stage. Products are read in transformation order, from left to right. Each directed edge permits zero, one or two mandatory whole-genome duplications, with A around them:

```
0 WGDs:  A
1 WGD:   A D A
2 WGDs:  A D A D A
```

D maps every ordinary CN c to min(2*c, 8), preserving X. One D costs one for the entire genome, across both alleles and all chromosomes. The limit of two applies to each directed edge, not to the whole tree or to each chromosome separately.

Example:

```
parent:  4 X 5
one D:   8 X 8       total cost = 1
```

Capping makes the 5 -> 8 part legal under a mandatory doubling. The previous PDF described the parent model; this edition explicitly describes saturation. Ordinary gains must still stay within the 0-8 alphabet.

Code: medicc/cext/full_fst/common/src/wgd_model.hpp

What a finite-state transducer remembers

An FST is a graph of legal transformations. Each arc reads an input CN, writes an output CN and charges a cost. A complete path transforms an entire profile. Among paths for the same endpoints, the smallest cost wins.

Primitive states record enough history to continue an interval without charging its opening cost again. In the literal loss primitive, states count active unit loss intervals. Increasing the active count opens events; continuing it is free.

One loss over two bins, illustrated:

```
state 0 -- read 2 / write 1 / pay 1 --> state 1
state 1 -- read 2 / write 1 / pay 0 --> state 1
```

At X, interval state resets. This is why adjacent bins can share an event but separate chromosomes cannot share a segmental loss.

An acceptor is simpler: it reads just one string. A weighted acceptor stores possible profiles and their costs. Ancestor messages use this representation.

Code: medicc/factory.py; medicc/cext/full_fst/*/assets/ancestral_parts.txt

Four operations to keep separate

Let T(p,c) be the minimum allowed directed event cost from parent p to child c. It can be infinite. Direction matters: a lost allele cannot be regained.

The pair distance minimizes over a shared ancestor z:

```
K(x,y) = min_z [T(z,x) + T(z,y)]
```

K is symmetric. It does not mean that x is the ancestor of y. For example, T(0,1) is infinite, but K(0,1) is 1: ancestor 1 can lose one copy toward 0.

The other operations take messages rather than two fixed endpoints:

- string(x) creates a message accepting only x, with cost zero.
- join(left,right) returns every locally minimum-cost ancestor word.
- resolve(parent,message) selects one word minimizing directed cost plus the word's stored message cost, conditional on the fixed parent.

Hint: a scalar distance, a set of ancestors, and one chosen ancestor are three different outputs. An optimization safe for one need not be safe for the others.

Code: medicc/full_fst/pair_scorer.py; medicc/full_fst/model.py

Fixed-string distances and chromosome splitting

For a pair of observed profiles, choose global WGD counts k and l first. Each is in {0,1,2}. Only then can X-delimited blocks be scored independently. Let C_i(k,l) be block i's optimum excluding the fixed WGD charge:

```
K(x,y) = min over k,l [k + l + sum_i C_i(k,l)]
```

Do not move the minimum inside the sum: that would allow each block to choose its own number of whole-genome duplications.

The production pair engine solves short blocks first. Exact costs from cheap blocks, plus lower bounds on unsolved blocks, may rule out a WGD combination before its expensive blocks need solving. Output genome order is unchanged. Feasible witnesses provide upper bounds. Repeated block scores can be cached.

Identical adjacent endpoint column pairs may be collapsed for scoring, keeping X boundaries. Ancestral messages retain the original bin grid. A bound-limited search failure is not cached as an exact distance.

Code: medicc/cext/full_fst/direct_pair/src/pairwise.cpp

A message is a dictionary without enumeration

For intuition, imagine a message as a Python dictionary:

```
{profile: subtree_cost, ...}
```

A real message is a weighted deterministic finite automaton (DFA). Prefixes and suffixes are shared instead of materializing every word. Its total word cost is a common offset plus arc weights plus a final weight. Rejected words have infinite cost. General imported messages may have unequal word weights.

A leaf is {observed_profile: 0}. A prune-zero join keeps only words tying at its local optimum. Normalized output can store that common cost in the offset and use zero residual weights on the surviving paths.

Do not replace a message by independent per-bin choices. A language containing only 11 and 22 does not also contain 12 and 21. Those cross-bin correlations matter both to search and to safe chromosome splitting.

message.language() enumerates words only for tiny examples and tests. Use message.size and native search statistics when inspecting real messages.

Code: medicc/full_fst/message.py; common/src/automata_types.hpp

The exact local join objective

For retained child messages M_left and M_right, compute:

```
F(z) = min over x,y [T(z,x) + M_left(x)
                   + T(z,y) + M_right(y)]
best = min_z F(z)
output(z) = F(z) if F(z) == best else infinity
```

The x and y choices range only over words accepted by the child messages. All ancestor words tying at best survive, including ties from different global WGD combinations. The standard backend calls this prune_weight=0.

This is exact for the retained inputs. It is not an unpruned dynamic program for the full-tree objective: a locally more expensive word could have been cheaper once connected to a higher parent, but it has already been discarded. The top-down pass cannot restore discarded words.

Hint: when reviewing a claim of exactness, ask whether it concerns a pair score, a local weighted-language join, or the whole-tree optimization problem.

Code: medicc/full_fst/model.py::_join; ancestors/src/nfa_engine.hpp

Worked example: why both ancestors survive

Join leaf messages for one-bin profiles 1 and 2. Two candidates tie:

```
ancestor z    T(z,1)    T(z,2)    total
     1           0         1         1
     2           1         0         1
```

Other CNs do not improve the total. The join's full language is therefore {'1': 1, '2': 1}, with offset 1. Selecting just one at this stage would lose an alternative that may matter when its parent is known.

Now resolve the same message under two different parent profiles:

```
resolve('1', message) -> ('1', 1)
resolve('2', message) -> ('2', 1)
```

The returned cost includes the stored message cost; it is not just the edge cost. Both selected edges in these examples cost zero. Final branch scores are calculated between the chosen endpoint profiles.

These values were checked against the current native engine for this edition. Page 23 gives a small script that reproduces them after building the package.

Code: medicc/full_fst/model.py; medicc/full_fst/message.py

Lazy composition avoids the full history graph

Conceptually, compose all factors in a permitted history and search that graph. In practice, building the entire product can consume enormous memory before we know which states are relevant. The implementation creates states on demand.

A search state records the column, the left and right primitive-state tuples, and positions in the child-message acceptors. One step chooses a shared ancestor symbol, finds compatible transitions on both sides, and advances.

Conceptual expansion:

```
for ancestor_symbol in allowed_symbols[column]:
    for left_edge in compatible_left_edges(ancestor_symbol):
        for right_edge in compatible_right_edges(ancestor_symbol):
            consider(next_tuple, left_edge.cost + right_edge.cost)
```

The same ancestor symbol must feed both histories. Each side may write a different child symbol. Impossible or provably too expensive combinations are skipped. Many histories can reach the same tuple, so prefix costs are merged.

Code: ancestors/src/nfa_engine.hpp::Side, StateKey, SearchState, search

Lower bounds, upper bounds and safe pruning

For a search state, g is the prefix cost already paid; h is a certified lower bound on the remaining cost. U is the cost of a feasible complete solution. Offsets and the fixed WGD charge must be included consistently in comparisons.

For an all-tie ancestor search:

```
if g + h > U:
    discard_state()
# Equality remains eligible: it could encode another best ancestor.
```

An initial U comes from chosen child words and their exact pair score plus stored costs. A greedy walk through legal transitions may tighten U. A failed greedy walk cannot certify infeasibility; the exact search still decides.

Bounds use suffix costs, chromosome information and, where supported, a relaxed sibling problem. Combine overlapping lower bounds with max, not their sum. Disjoint cost components can be added when the accounting justifies it.

Scalar pair scoring only needs the best number, so it can skip paths merely tying an already feasible incumbent. Do not copy that rule into an all-tie join.

Code: common/src/feasible_path.hpp; ancestors/src/nfa_engine.hpp

Find the score, then recover every tying word

The forward A* pass computes best prefix distances. Boost.Graph manages the ancestral priority queue and relaxation. A forward edge that cannot improve a destination's prefix distance need not be handed to Boost, even if it ties.

That does not by itself collect the ancestor language. A separate backward pass regenerates transitions, computes exact suffix costs on the surviving region, and retains tight arcs:

```
tight arc (s -> t):
    best_prefix(s) + arc_cost + best_suffix(t) == optimum
```

All labels on tight paths survive, including equal-cost alternatives skipped as redundant forward relaxations. The backward pass must not use the forward strict-improvement filter to drop these labels.

The retained nondeterministic acceptor is determinized; tying WGD languages are united. OpenFST handles determinization, weighted union and epsilon removal. Specialized acyclic minimization and normalization compact the messages further. This compresses optimal alternatives without selecting a single profile.

Code: ancestors/src/nfa_engine.hpp; common/src/openfst_language.cpp

Exact compact AD factors

The history A D A D A can be grouped as (AD)(AD)A without changing the weighted relation. OpenFST composes A with capped D, then reduces that small factor using paired-label encoding, determinization and minimization.

The checked-in compact AD factor has 74 states and 3,108 arcs; the A factor has 212 states and 15,688 arcs. The generator checks weighted equivalence of the reduced factor and its canonical text export, including X resets.

This is a factor representation change, not an approximate event model. The production bottom-up joins and pair scoring use it; they do not eagerly construct the full two-WGD history relation.

The downward resolver keeps the original A,D,A primitive ordering. Equivalent factors can have different state orders and select different representatives among equal-cost words. Preserving the original order preserves that historical tie choice, even though the set of optimal words is unchanged.

Suffix-table preparation also indexes arcs by observed child output label, so sparse child messages visit only compatible arcs. The exact recurrence, child weights, final weights and separator resets are preserved.

Code: scripts/generate_capped_ad.py; tests/check_resolve_tie.py

A smaller sibling problem guides the search

Independent left/right bounds may look cheap by relying on incompatible ancestor symbols. The sibling pattern bound solves a relaxed problem that still requires both sides to consume the same ancestor symbol at each bin.

Each primitive state is mapped to one of four classes according to whether zero-cost outgoing arcs allow decreases, increases, both, or neither. All real arcs are projected; duplicate abstract arcs retain their minimum cost.

The bound uses per-column child-symbol masks, ignoring correlations and nonnegative child weights. Every real suffix still maps to an abstract path of no greater cost. Thus the abstract optimum is a safe lower bound.

The table has 16 paired states for 0/0 WGDs, 64 for 0/1 or 1/0, and 256 for 1/1. If either side has two WGDs, the existing certified bounds and exact search remain in use. Those histories are not removed.

This bound is used in ancestral joins. The production fixed-string distance engine does not use this sibling pattern table.

Code: ancestors/src/pattern_bound.hpp; doc/search_explosion_optimization.rst

Exact fallback for unusually expensive joins

After 250,000 expansions in a fixed-WGD join search, the engine checks whether it can restart using independent segments. This number selects an algorithm; it is not a state limit, timeout, beam width or permission to approximate.

A safe cut must follow an explicit X. Both child DFAs must have a single state at that layer; every primitive must reset to its start at X, with zero final cost there. These checks prevent splitting correlated child languages wrongly. If no safe cut exists, the original search continues.

For fixed global WGD counts, solve each safe segment exactly, retain all its optimal words, then concatenate segment languages in their original order. Short segments are solved first to tighten the remaining budget. The WGD charge and child offsets are applied once to the complete result.

The original explored graph is released before the restart. Segment searches do not restart recursively. A segment itself may still be expensive: this is not a polynomial-time guarantee or a promise that every large case becomes fast.

Code: ancestors/src/segmented_join.hpp; doc/ancestral_outlier_optimization.rst

Two tree passes, then directed branch scores

Think of ordinary tree traversal, with native message operations at each node. This is pseudocode; it omits scheduling and ownership details:

```
for leaf in leaves:
    message[leaf] = model.string(observed[leaf])
for v in internal_nodes_bottom_up:
    message[v] = model.join(message[left[v]], message[right[v]])

for v in internal_nodes_top_down:
    p = normal if v == root else profile[parent[v]]
    profile[v], cost = model.resolve(p, message[v])
```

Observed leaf profiles remain fixed. The root is resolved relative to the normal profile; later internal nodes are resolved relative to their selected parent. The workflow scores final branches from their selected endpoints.

Workers run ready nodes after dependencies complete. Each worker has its own mutable Model; child messages are read-only inputs. Optional branch parallelism shares a worker budget with tree traversal to control concurrent native work.

Code: medicc/full_fst/scheduler.py; medicc/full_fst/core.py

Topology search and reuse

Without a supplied tree, FastME constructs the topology from pair distances. The workflow uses its BME/NNI/SPR configuration with seed 104000. Optional MEDICC NNI subsequently proposes local topology rearrangements and evaluates them using the full-model ancestral backend.

A changed topology does not invalidate every message. Bottom-up messages for unchanged subtrees can be reused. Top-down reconstruction can reuse work only when the relevant subtree and parent context remain valid.

Cached messages share ownership. Closing a cached native handle too early would invalidate another candidate tree's input. Read the explicit cache and lifetime rules before changing this part of the code.

FastME names at or above 64 UTF-8 bytes are mapped to unique short IDs and restored after inference. This is an input compatibility fix, not a change to the event model. The output mapping is recorded in fastme_taxa.tsv.

Code: medicc/full_fst/ancestors.py; medicc/core.py; medicc/full_fst/core.py

From branch scores to optional event histories

--events runs after ancestor reconstruction. For each selected branch, the native pair engine chooses an optimal WGD count and intermediate profiles. OpenFST shortest-path calls decompose each constrained A or AD stage into literal loss/LOH and gain transitions. Active interval counts become unit events.

A verified example with separated one-bin blocks:

```
1X1 -- gain left --> 2X1 -- gain right --> 2X2
    -- WGD --> 4X4 -- WGD --> 8X8
four unit events; exact directed cost = 4
```

The caller returns one optimal history, not all histories. Ties prefer fewer WGDs, then one native/OpenFST path. Replay must reach the child, and the unit event count must equal the branch score; otherwise the caller raises an error.

Shortened columns expand back to original bins before coordinates are emitted. Optional BED annotations summarize regions with strictly more than 90% event coverage. They do not change raw event counts or reconstruction; use the ordered _copynumber_events_df.tsv for replay.

Code: medicc/full_fst/events.py; doc/full_fst_review.rst

Read the Python/C++ boundary without getting lost

Python owns input/output, tree traversal, NNI orchestration and native handles. The full-FST API uses ctypes over a C ABI. The inherited FSTlib wrappers use Cython. The numerical search kernels are C++17.

A small translation key:

- const DFA &x borrows a read-only object without copying it.
- vector<State> is an owned array; integer IDs survive reallocation.
- auto &x refers to the stored object; auto x makes a copy.
- [&](...) { ... } is a callback borrowing surrounding variables.
- std::move permits transferring storage; do not assume old contents remain.
- void* at the C ABI is an opaque handle; Python never dereferences it.

Model owns the native model allocation. Message owns its DFA and keeps its originating Model alive. close() is idempotent. Returned native strings are copied to Python bytes by ctypes. Do not close messages still held by NNI caches.

Code: medicc/full_fst/_bindings.py; model.py; message.py

What the libraries do, and what remains specialized

OpenFST performs ancestral determinization, weighted union and epsilon removal. It also generates the compact AD factor and decomposes optional event stages. Boost.Graph provides A* for ancestral joins and conditional traceback.

MEDICC supplies the biological model, admissible bounds, lazy composition, chromosome/WGD bookkeeping, tight-path language extraction, specialized state indexing, and acyclic integer minimization/normalization. Fixed-profile pair search remains specialized; it is not the Boost ancestral search.

The independent reference_pair backend supports comparisons. The normal production backend is direct_pair. Shared names do not mean every backend uses the same implementation or optimization.

The checked-in tables and generated headers are model assets. Reading them line by line is unnecessary for a first review. Start with the routines that consume them. The filename budget_2m_core.hpp is historical: it does not impose a two-million-state search limit. Here, "uncapped search" means no arbitrary state cutoff; copy numbers are still capped at 8.

Code: common/src/openfst_language.cpp; common/src/boost_search.hpp

What has been checked, and what exactness does not mean

The repository contains small-case checks using an independent literal-FST oracle, weighted ancestor-language comparisons, event replay, binding checks and workflow regressions. See README.md and the individual optimization notes for historical validation counts and benchmark conditions.

Useful checks after building:

```
python -B tests/check_pairs.py
python -B tests/check_oracle.py
python -B tests/check_ancestor_bounds.py
python -B tests/check_resolve_tie.py
python -B tests/check_events.py
```

Compare full weighted languages when testing all-tie joins. Comparing one chosen profile alone can miss a lost tie. A correct scalar cost also does not guarantee the same historical representative in the downward pass.

Exactness concerns the specified bounded model and retained-message objective. It does not guarantee the true biological history, a global optimum over all trees/ancestors, or bounded runtime on every input. This is an experimental implementation; historical test coverage is not a substitute for code review.

Reproduce the tiny example

This executable example needs a built checkout and its dependencies. Reading the guide and PDF does not. Run it from the repository root:

```
from medicc.full_fst.model import Model
from medicc.full_fst.pair_scorer import Model as PairScorer

with PairScorer() as scorer:
    assert scorer.score('4X5', '8X8', directed=True) == 1
    assert scorer.score('1X1', '8X8', directed=True) == 4

model = Model()
left, right = model.string('1'), model.string('2')
joined = model.join(left, right)
try:
    assert joined.language() == {'1': 1, '2': 1}
    assert model.resolve('1', joined) == ('1', 1)
    assert model.resolve('2', joined) == ('2', 1)
finally:
    joined.close()
    left.close()
    right.close()
    model.close()
```

Hint: change the fixed parent and predict the result before running it. Keep language() enumeration to tiny messages; the production graph may encode far more words than you can reasonably print.

Your offline reading route

Read source in this order, asking one concrete question at each step:

- medicc/full_fst/core.py: how do inputs become profiles and a topology?
- scheduler.py: when are bottom-up and top-down operations ready?
- model.py and message.py: what is minimized and what is retained?
- pair_scorer.py: is this a directed score or a shared-ancestor distance?
- Native ancestors/src/nfa_engine.hpp: which states can be discarded?
- Native ancestors/src/segmented_join.hpp: when is a cut independent?
- Native direct_pair/src/pairwise.cpp: where is WGD charged only once?
- medicc/full_fst/events.py: how are histories decomposed and replayed?

Python filenames without a prefix above are under medicc/full_fst; native paths start at medicc/cext/full_fst. The deeper ownership guide is doc/full_fst_review.rst. The other RST notes retain change-specific evidence.

All source, model tables and this guide/PDF are in Git. External benchmark inputs, results and experiment drivers are not. The saved conda environment is Linux-specific; macOS execution needs a separate dependency/build setup.

To rebuild this PDF with Python and Matplotlib (no native MEDICC build needed), run python scripts/build_algorithm_pdf.py. The script reads this RST file.